



Agentic AI Task

AI Governance — Problem Statements

June 2026

Prepared by: Aivar Innovations

Introduction

Your Task

Pick any one problem statement and implement a production-ready solution. You are expected to deliver a working system — not a prototype, not a slide deck, not a demo script.

Production Readiness Matters

Solutions will be evaluated on production readiness. A working solution that runs only on localhost will score lower than one that is deployed and governs real AI workloads in a cloud environment. The closer your solution is to something an enterprise could actually adopt, the higher it will be rated.

The following are examples of what production readiness means in practice:

- A solution deployed on AWS (Lambda, ECS, EKS, or equivalent) that can govern agents also hosted on AWS scores higher than one running on localhost.
- A solution that handles concurrent requests, persists state, and exposes a usable API scores higher than one that runs as a single script.
- A solution with logging, error handling, and a basic health check scores higher than one without.
- A solution that connects to at least one real LLM provider (OpenAI, Anthropic, AWS Bedrock, etc.) scores higher than one using mocked responses only.

Extra points are awarded for deployment quality, integration breadth, and the degree to which the solution could be plugged into a real enterprise AI stack without significant rework.

Problem Statement Format

Each problem statement contains the following sections:

- Context — the real-world pain this problem addresses and why current tools fail to solve it.
- The Challenge — one sentence stating exactly what you are building.
- What to Build — the concrete components and capabilities your solution must include.
- Success Criteria — the testable conditions your solution must satisfy to be considered complete.
- Bonus — optional extensions that demonstrate depth and earn extra credit.

Problems are grouped into ten capability units. Each unit represents a distinct layer of the AI governance stack. You may pick any problem from any unit.

Unit 1 AI Asset Inventory & Registry

Before anything can be governed, every AI system — model, agent, tool, API connection, dataset — must be discovered, catalogued, and classified. Current inventories are manual, stale, and incomplete. These problems address the foundational visibility gap.

PS-1.1

The AI Asset Crawler

Context

An enterprise has dozens of teams deploying LLM-powered apps and agents. Nobody has a complete list of what is running. The central IT team manually maintains a spreadsheet that is weeks out of date the moment it is published. When an incident occurs, no one can quickly answer: what AI systems touched this data?

The Challenge

Build an event-driven AI asset discovery agent that automatically detects and registers AI systems as they are deployed — without requiring manual intake.

What to Build

- A lightweight crawler or webhook listener that monitors a target environment (pick one: AWS account via CloudTrail events, a Kubernetes cluster via API server events, or a GitHub organisation via push/deploy webhooks).
- When it detects a new AI-related deployment (an LLM API call pattern, a new Lambda with an AI SDK dependency, a new container image referencing an AI model), it automatically creates or updates a record in a simple asset registry.
- Each record captures: asset name, type (model / agent / tool), owner inferred from deployment metadata, first-seen timestamp, last-active timestamp, and a basic risk tier (low / medium / high) inferred from the asset's data access scope.
- The registry is exposed as a queryable API.

Success Criteria

- Deploy three sample AI services in the target environment; the crawler detects all three within 60 seconds without manual input.
- Registry API returns a current asset list with correct metadata for each discovered asset.
- A fourth hidden deployment simulating a shadow AI service is also detected and flagged.

Bonus

- Add a diff view showing what changed in the registry in the last 24 hours.
- Send an alert when a new asset appears that has no declared owner.

PS-1.2

MCP Server Risk Scanner

Context

Model Context Protocol (MCP) servers are the tool layer that AI agents connect to. An agent given access to an MCP server can call any tool that server exposes — read files, query databases, call external APIs. As of early 2026, thousands of MCP servers are reachable on the public internet without authentication. Most organisations have no inventory of which MCP servers their agents are connecting to, let alone whether those servers are safe.

The Challenge

Build an MCP server inventory and risk scanner that discovers which MCP servers an agent fleet is connecting to and produces a risk assessment for each.

What to Build

- A proxy layer that sits between an agent and its MCP connections, passively recording every MCP server URI the agent connects to — no modification to agent behaviour, observe only.
- A scanner that for each discovered MCP server checks: Is authentication required? What tools does it expose? Does it expose any tools that write data, delete data, or call external APIs? Is the server reachable from the public internet?
- A risk card per server: server URI, tool list, authentication status, write-capability flag, public exposure flag, and overall risk tier.
- A simple dashboard showing the fleet's MCP exposure surface.

Success Criteria

- Run three sample agents with different MCP server configurations; scanner discovers all connections and produces a risk card for each.
- Correctly identifies an unauthenticated, publicly reachable MCP server as high risk.
- Risk cards are accurate based on actual tool metadata returned by the server.

Bonus

- Auto-generate an OPA policy stub that restricts agent access to flagged high-risk MCP servers.

PS-1.3

The Governance Graph Builder

Context

An agent is a composition: an LLM, an orchestration layer, a set of tools, a memory store, and data sources it can access. Today these components are tracked in separate systems — the LLM in a model registry, the tools undocumented, the data sources in a data catalogue. When something goes wrong, no one can reconstruct what was connected to what.

The Challenge

Build a governance graph that represents agents, models, tools, users, and policies as connected nodes — making the full composition of any agent queryable in one place.

What to Build

- A graph data model with node types: User, Agent, Model, Tool, DataSource, Policy (use Neo4j, NetworkX, or any graph library).
- An ingestion layer that builds the graph from three sources: an agent config file, runtime logs from a sample agent run, and a manually authored policy document.
- Queries that answer: What tools does Agent X have access to? Which agents are using Model Y? Which agents can access DataSource Z? Is there any agent with no policy attached?
- A visual graph render of the result.

Success Criteria

- Given three sample agent configs of increasing complexity, the graph is correctly built and all queries return accurate results.
- The orphan agent query (agents with no policy) returns correct results when one agent is intentionally left policy-free.
- Graph updates correctly when an agent config changes.

Bonus

- Add a blast radius query: given a compromised tool, which agents and users are potentially affected?

Unit 2 Identity & Access Control

The principal in AI systems is often not a human. Enterprise IAM was built for human users. These problems address the identity and permission gap that emerges when the primary actors are non-human agents operating with delegated authority.

PS-2.1

Agent Identity Card

Context

When a human joins a company they get an identity: a user account, a role, an access scope, and a review cadence. When an AI agent is deployed it typically gets an API key — no role, no scope, no review, no expiry. That API key often has the same permissions as the developer who created it.

The Challenge

Build an Agent Identity Management system that provisions and manages machine identities for AI agents with the same rigour applied to human user accounts.

What to Build

- An agent registration flow: input an agent name, purpose description, owning team, and requested tool scopes; output a scoped time-bounded credential and an identity record.
- Each identity record includes: agent ID, owning team, creation date, expiry date, approved tool scopes, and a status (active / suspended / decommissioned).
- A credential rotation mechanism: agents can request a new credential before their current one expires; old credential is revoked.
- A quarterly access review simulation: list all active agents, flag any that have not made an API call in 30 or more days as stale, and generate a review report for the owning team.
- An auto-revoke trigger: if an agent's expiry date passes without renewal, its credentials are automatically deactivated.

Success Criteria

- Register three agents with different scopes; each receives a working scoped credential.
- Credentials correctly enforce scope — an agent with read-only scope cannot perform write operations.
- Stale agent report correctly identifies agents inactive for 30 or more days.

- Auto-revoke works: an expired agent's credential is rejected at call time.

Bonus

- Integrate with a real IAM provider (Okta or Auth0 free tier) to issue actual OIDC tokens for agents.

PS-2.2

Tool Permission Enforcer

Context

An agent has access to a CRM tool. That tool can read customer records, write to customer records, and delete customer records. The agent's policy says it is a customer support agent — it should only be able to read records for the customer currently in the session. But at the infrastructure level it has full CRM access. Nothing enforces the narrower scope.

The Challenge

Build a tool permission proxy that enforces fine-grained, scope-aware access control on every tool call an agent makes.

What to Build

- A proxy layer that sits between an agent and its tools (implement as a thin wrapper around a sample tool set such as a mock CRM API with read, write, and delete operations).
- A permission manifest per agent: agent_id, tool_name, allowed_operations (e.g., read only), and data_scope (e.g., customer_id must equal session customer_id).
- The proxy evaluates every tool call against the permission manifest before forwarding it to the actual tool.
- Calls that violate the manifest are blocked and logged with a reason; calls within scope are forwarded transparently.
- A session context injector: the proxy reads session metadata and uses it to evaluate dynamic data scope rules.

Success Criteria

- Agent with read-only scope successfully reads a customer record.
- The same agent's attempt to write to the record is blocked at the proxy with an audit log entry.
- An attempt to read a different customer's record outside session scope is blocked.

- All blocks and passes are logged with timestamp, agent ID, tool, operation, and outcome.

Bonus

- Add a real-time alert when an agent attempts an operation outside its declared scope more than three times in a session, indicating potential probing behaviour.

PS-2.3

Delegation Chain Governor

Context

Agent A is triggered by a user. It has permission to read financial records. Agent A calls Agent B to do analysis. Agent B calls Agent C to fetch additional data. Agent C now has effective access to financial records — but nobody explicitly granted it that access. The permission scope silently expanded across three delegation hops.

The Challenge

Build a delegation chain enforcement layer that propagates a scoped permission token through multi-agent calls and prevents scope expansion at each hop.

What to Build

- A delegation token format: a signed, scope-constrained token issued at the start of a task (originating user, task ID, max allowed scope, current delegation depth, expiry).
- A token propagation mechanism: when Agent A calls Agent B, it passes a derived token with scope equal to or narrower than its own — it cannot issue a token with broader scope than it holds.
- An enforcement interceptor: before any agent executes a tool call, it validates its delegation token; if the token's scope does not permit the operation, the call is rejected.
- A delegation audit log: the full chain of delegations for a task, reconstructed from token payloads.

Success Criteria

- A 3-agent chain correctly propagates a read-only financial token; write operations by Agent C are rejected.
- An attempt by Agent A to issue a token with broader scope than its own is rejected by the token system.
- Delegation audit log correctly reconstructs the full call chain from token payloads.
- Token expiry is enforced: tokens older than their TTL are rejected mid-chain.

Bonus

- Implement scope shrinkage: Agent A can restrict the scope further before passing to Agent B, for example from `read:all_financials` to `read:summary_only`.

Unit 3 Runtime Guardrails & Policy Enforcement

Policy on paper is not policy. These problems address the enforcement gap — making governance operational at the moment of execution rather than in a compliance document reviewed quarterly.

PS-3.1

The Action Guardrail

Context

Every commercial guardrails platform filters LLM text input and output. None of them govern what an agent does after the LLM produces output. An agent can generate a perfectly clean, non-toxic response that then instructs a tool to delete ten thousand database records. Current guardrails pass this through without intervention.

The Challenge

Build a guardrail layer that operates on agent actions — not just text — enforcing policy at the moment a tool call is about to execute.

What to Build

- A pre-execution action evaluator: before any tool call is dispatched, evaluate it against a policy ruleset.
- Policy rules expressed in a simple declarative format (YAML or JSON), supporting at minimum: block (reject the call), require_hitl (pause for human review), and log_and_allow (execute but create an audit record).
- Example rules: block any database delete where record count exceeds 100; require HITL for any email sent to an external domain; log and allow any read of a path containing the word confidential.
- A simulation harness: run a sample agent through a scenario that triggers each of the three outcome types.

Success Criteria

- All three outcomes fire correctly on matching tool calls.
- A bulk delete of 500 records is blocked; a delete of five records is allowed.
- An email to an external domain pauses for HITL; an internal email goes through.
- Audit log captures every evaluated action with outcome and matched rule.

Bonus

- Add a dry run mode: the agent runs normally but all actions are simulated and policy violations are reported without executing.

PS-3.2

Prompt Injection Behavioral Detector

Context

Pattern-matching for prompt injection catches naive attacks. Sophisticated injections arrive embedded in retrieved documents, tool outputs, or structured API responses — the injection looks like legitimate data until it hijacks the agent's next action. Current tools miss these because they look at prompt syntax, not at whether the agent's subsequent behaviour changed.

The Challenge

Build a behavioral anomaly detector for prompt injection that detects successful injections by observing what the agent does next, not by inspecting the prompt.

What to Build

- A baseline profiler: run a sample agent through 20 to 30 normal task scenarios and record its behavioural fingerprint — which tools it calls, in what order, with what parameter patterns.
- An anomaly scorer: given a new agent run, compare its tool call sequence and parameter patterns against the baseline and produce an anomaly score.
- An injection simulation: craft three prompt injection payloads embedded in tool outputs (not in the user prompt) that cause the agent to deviate from its normal behaviour.
- Demonstrate that your detector flags the injected runs as anomalous while normal runs score low.

Success Criteria

- Baseline correctly established from 20 or more normal runs.
- All three injected runs produce anomaly scores significantly above the normal range.
- At least two of twenty normal runs produce higher scores but remain below the flagging threshold, demonstrating calibration rather than a binary check.
- Detection happens within one agent turn of the injection taking effect.

Bonus

- Add a suspicion accumulator: anomaly score accumulates across turns within a session; a persistent low-level anomaly is flagged even if no single turn crosses the threshold.

PS-3.3

Cross-Provider Guardrail Policy Engine

Context

An enterprise runs three LLM providers: OpenAI for customer-facing features, Anthropic for internal tools, and a self-hosted model for sensitive workloads. Each has its own guardrails interface. The same policy — block PII in all outputs — must be re-implemented three times in three proprietary formats, and the self-hosted deployment may have no guardrails at all.

The Challenge

Build a provider-agnostic guardrail policy engine that applies a single policy definition uniformly across multiple LLM providers.

What to Build

- A unified policy definition format (YAML or JSON) that expresses guardrail rules independent of any provider — for example, check PII in output with action redact, or check toxic content with action block at threshold 0.8.
- A policy executor that translates and applies the same policy to at least two different LLM providers.
- Built-in checks: PII detection using an open-source NER model or regex, toxicity scoring using a lightweight classifier, and topic denial using keyword plus semantic similarity matching.
- A unified audit log across all providers using the same schema regardless of which provider handled the request.

Success Criteria

- A PII redaction policy defined once applies correctly to outputs from both providers.
- A toxic content block fires on the same input regardless of which provider generates the response.
- Audit log schema is identical across providers.
- Adding a third provider requires only a new provider adapter, not a rewrite of the policy logic.

Bonus

-
- Add policy inheritance: a base policy applies to all providers; a provider-specific overlay can restrict further but never relax the base policy.

Unit 4 Observability & Behavioral Monitoring

Monitoring that does not connect to governance is just logging. These problems address the gap between observing what an AI system does and taking a governed response when something changes.

PS-4.1

Agent Behavioral Baseline Builder

Context

You cannot detect abnormal agent behaviour without knowing what normal looks like. When a new agent is deployed, there is no established baseline — so the first weeks of production are effectively unmonitored from a behavioural standpoint. By the time a behavioural anomaly is detectable, the agent has been running ungoverned for weeks.

The Challenge

Build a system that automatically establishes a behavioural baseline for an agent at deployment time using synthetic traffic, then monitors production against it.

What to Build

- A synthetic scenario generator: given an agent's system prompt and tool list, automatically generate 50 diverse test scenarios that exercise the agent's expected behaviour space.
- A baseline recorder: run the agent against all synthetic scenarios and record the behavioural fingerprint — tool call frequency distribution, average response length, typical tool call sequences, data access patterns.
- A production monitor: as real requests come in, compute running statistics and compare against the baseline; flag sessions where behaviour deviates beyond a configurable threshold.
- A baseline drift detector: if production behaviour consistently shifts over time due to a model update or prompt change, auto-suggest a baseline refresh.

Success Criteria

- Baseline established from synthetic scenarios before any real traffic.
- Three production scenarios produce correct monitoring signals: normal (no flag), moderate anomaly (warning), and severe anomaly (alert).
- Baseline drift detected when a simulated model update changes the agent's tool call patterns.

Bonus

- Cluster baseline scenarios by intent type — information retrieval, data modification, communication tasks — and maintain per-cluster baselines for finer-grained monitoring.

PS-4.2

Governance-Linked Observability Pipeline

Context

A drift alert fires in a monitoring platform. An engineer sees it in Slack. Maybe they investigate; maybe they do not. The compliance team never hears about it. The governance record stays green. Three weeks later, during an audit, it emerges that a model was operating outside its documented performance envelope for 22 days while the compliance system showed no issues.

The Challenge

Build a pipeline that connects observability signals to governance actions — so that an anomaly in production automatically triggers a governance workflow, not just an engineering notification.

What to Build

- A signal ingester: consume monitoring events from a sample observability source (mock Prometheus metrics, OpenTelemetry spans, or a guardrail alert webhook).
- A governance trigger classifier: evaluate each incoming signal against a rule set that maps signal types to governance actions — drift above threshold opens a control re-assessment task; safety violation suspends the agent pending review; repeated guardrail blocks escalate to the compliance team.
- A lightweight governance state machine per agent: each agent has a governance status (green / under review / suspended / remediated); signals can advance or reset the status.
- A governance action log: every status change recorded with the triggering signal, timestamp, and current reviewer.

Success Criteria

- Five different signal types each trigger the correct governance action.
- Agent suspension on safety violation works: subsequent requests from a suspended agent are rejected with a governance-hold message.
- Governance action log is accurate and queryable.
- Status transitions follow state machine rules — a suspended agent cannot go directly back to green without passing through under review.

Bonus

- Add a governance SLA timer: if an agent stays in under review status for more than 48 hours without action, auto-escalate to a senior reviewer.

PS-4.3

Cross-Session Adversarial Pattern Detector

Context

A sophisticated attacker does not try to jailbreak an LLM in a single session. They probe across many sessions — sending slightly different variations, testing which inputs produce guardrail responses, gradually mapping the model's policy boundary. Each individual session looks benign. No current platform correlates behaviour across sessions to detect this pattern.

The Challenge

Build a cross-session behavioural analysis system that detects multi-session adversarial probing patterns against an LLM application.

What to Build

- A session event store: log all user interactions with an LLM application keyed by user identifier, with timestamps and guardrail outcomes.
- A pattern detector that identifies: guardrail boundary probing (a user sending semantically similar inputs that repeatedly trigger guardrail blocks); privilege escalation attempts (a user incrementally requesting higher-capability actions across sessions); systematic enumeration (a user requesting outputs for a large range of inputs in a structured pattern).
- A risk score per user identity that accumulates across sessions.
- An alert when a user's risk score crosses a threshold, with a summary of the detected pattern.

Success Criteria

- Detector correctly identifies a simulated boundary-probing user sending 20 sessions with gradual prompt variation all of which are blocked.
- Detector correctly identifies a simulated data-scraping user sending 50 sessions with sequential entity enumeration.
- A normal high-volume user with many sessions and diverse legitimate queries does not trigger false positive alerts.
- Risk score correctly accumulates and resets after a configurable inactivity window.

Bonus

- Cluster detected patterns by technique — probing, enumeration, escalation — and produce a threat summary card per flagged user.

Unit 5 Agentic AI Security

Agents can do things. Most governance tooling only governs what they say. These problems address the security gaps that are unique to autonomous agents operating with delegated authority over enterprise systems.

PS-5.1

The Agent WAF

Context

A Web Application Firewall sits between the internet and a web server, inspecting and filtering requests before they reach the application. There is no equivalent for agent tool calls. An agent can invoke any tool it has a connection to, with any parameters, with no inspection layer between the agent's decision and the tool's execution.

The Challenge

Build an Agent WAF — a policy-enforcing proxy between an agent and its tools that inspects, filters, and logs every tool invocation in real time.

What to Build

- A transparent proxy layer that intercepts all tool calls from a sample agent.
- A rule engine with at least these rule types: rate limit (agent may call Tool X no more than N times per minute); parameter validation (reject calls where parameter values match a blocklist or exceed size limits); data scope (reject calls that reference data outside the agent's declared scope); sequence rules (reject calls to Tool B unless Tool A was called first in this session).
- Each intercepted call is logged with timestamp, agent ID, tool, parameters sanitised, rule evaluation outcome, and final disposition.
- A real-time dashboard showing tool call traffic and block events.

Success Criteria

- Rate limit fires correctly after N calls within the window.
- Parameter blocklist catches a simulated injection attempt in a tool parameter.
- Out-of-scope data access is blocked.
- Sequence rule enforcement correctly blocks a tool called out of expected order.
- Dashboard updates in real time as calls flow through.

Bonus

- Add shadow mode: the WAF logs what it would have blocked without actually blocking, for safe calibration of new rules before enforcement.

PS-5.2

Inter-Agent Trust Verifier

Context

In a multi-agent system, Agent B receives instructions from Agent A. Agent B has no way to verify that the message actually came from Agent A and not an attacker, that Agent A is authorised to give Agent B that specific instruction, or that Agent A has not itself been compromised and is now issuing out-of-policy commands. Agent B executes whatever it receives.

The Challenge

Build an inter-agent trust protocol that allows agents to verify the legitimacy and authority of instructions received from other agents before executing them.

What to Build

- A signed instruction format: when Agent A sends a task to Agent B, it signs the instruction with its private key and includes its delegation token.
- Agent B's trust verifier: before accepting an instruction, Agent B verifies the signature, checks that Agent A's delegation token authorises it to issue this type of instruction, and checks that the requested action is within Agent B's own policy scope.
- An instruction audit trail: every accepted or rejected inter-agent instruction logged with the verification outcome.
- A compromised agent simulation: manually revoke Agent A's credentials mid-pipeline and show that Agent B correctly rejects subsequent instructions from Agent A.

Success Criteria

- Valid signed instruction from authorised Agent A is accepted by Agent B.
- Unsigned instruction simulating man-in-the-middle is rejected.
- Instruction that exceeds Agent A's delegation scope is rejected even if signed correctly.
- Revoked credential instruction is rejected within one verification cycle of the revocation.

Bonus

- Implement a trust reputation score: agents that have previously issued invalid or rejected instructions accumulate a negative score; high-scoring agents trigger heightened scrutiny.

PS-5.3

Semantic Data Exfiltration Detector

Context

Standard DLP tools detect exfiltration by pattern-matching on known data formats such as credit card numbers or email addresses. An AI agent can exfiltrate data semantically: asking the LLM to paraphrase a sensitive document, summarise a confidential record, or answer questions that reconstruct protected information piece by piece. No DLP tool catches this because the output does not match any known pattern.

The Challenge

Build a semantic data exfiltration detector that identifies when an agent's outputs are carrying sensitive information in non-standard form.

What to Build

- A reference data vault: a small set of protected documents or records such as fake employee records, mock financial data, or synthetic medical records.
- An embedding-based similarity scorer: for every agent output, compute its semantic embedding and compare against the embeddings of protected documents; flag outputs with high similarity even when the text is paraphrased.
- A factual overlap detector: use an LLM to evaluate whether an output contains specific facts that are only present in the protected vault, even when expressed differently.
- A test suite: 10 normal outputs with low overlap, five paraphrased vault contents with high overlap, and five borderline cases.

Success Criteria

- Similarity scorer correctly ranks paraphrased vault content higher than unrelated outputs.
- Factual overlap detector correctly identifies four of five paraphrased cases as vault-derived.
- False positive rate on normal outputs is below 20 percent.
- Detection works even when the agent deliberately obfuscates by reconstructing facts without directly quoting the record.

Bonus

-
- Add a data lineage tag system: protected documents are tagged at ingest; any output that semantic-matches a tagged document carries the tag in its audit metadata.

Unit 6 Compliance & Regulatory Management

Compliance that lives in a document is not compliance. These problems address the gap between what regulators require and what is actually enforced — with a focus on LLM and agent-specific obligations that existing compliance platforms have not yet solved.

PS-6.1

Agent Compliance Card Generator

Context

EU AI Act Article 13 requires transparency documentation for high-risk AI systems. Model cards exist for ML models. Nothing equivalent exists for agents — a document that captures what an agent does, what tools it can invoke, what data it accesses, what decisions it can influence, what human oversight is in place, and what its risk classification is. Compliance teams are generating this manually in Word documents.

The Challenge

Build an automated agent compliance card generator that produces a structured, regulation-aligned transparency document for any agent from its configuration and runtime data.

What to Build

- An agent card schema covering: purpose and scope, LLM used with version, tool inventory with operations and data access, data sources, decision authority (informational / advisory / autonomous), human oversight mechanisms, risk classification, known limitations, and incident contact.
- A card generator that populates the schema by parsing an agent config file, a tool manifest, and a sample run trace.
- A regulation mapper: annotate each card field with the regulatory obligation it satisfies — EU AI Act Article 13, NIST AI RMF Govern function, ISO 42001 clause.
- A completeness checker: flag any card fields that are missing or contain placeholder values.
- Export as structured JSON and as a human-readable document.

Success Criteria

- Given three agent configs of increasing complexity, three accurate agent cards are generated.
- Regulation mapper correctly annotates all fields with at least one framework reference.
- Completeness checker correctly flags an intentionally incomplete card.

- Human-readable output is clear enough for a non-technical compliance reviewer to understand.

Bonus

- Add a card diff view: when an agent is updated, show which card fields changed and flag any changes that require a regulatory re-assessment.

PS-6.2

Runtime-to-Compliance Bridge

Context

A compliance platform says an agent's guardrails are active and compliant. In production, the guardrail is misconfigured and has been silently failing for two weeks. The compliance system has no way to know because it gets its data from periodic self-assessments, not from live runtime signals. The result is a compliance record that is demonstrably false.

The Challenge

Build a bridge that continuously syncs runtime enforcement state into a compliance record — so compliance status reflects what is actually running, not what was documented at deployment.

What to Build

- A compliance record schema for an agent: `guardrails_active`, `last_violation_timestamp`, `pii_redaction_enabled`, `audit_logging_enabled`, `hitl_configured`, and `overall_compliance_status` (green / amber / red).
- A runtime telemetry collector: poll or receive webhooks from a running agent's guardrail system and audit logger.
- A sync engine: update the compliance record every five minutes based on live telemetry; if a guardrail check that should be active has produced zero events for 24 hours, flag it as potentially inactive.
- A compliance status timeline: show how compliance status changed over the last seven days with the triggering event for each change.

Success Criteria

- Compliance record correctly reflects active guardrails when all systems are functioning.
- A simulated guardrail failure triggers a status change from green to amber within 24 hours and red within 48 hours.
- Timeline correctly shows status transitions and timestamps.

-
- Manually re-enabling the guardrail restores the compliance status to green in the next sync cycle.

Bonus

- Add a compliance SLA: if an agent stays in amber or red status for more than a configured number of hours, auto-generate an incident record with the remediation steps.

Unit 7 Audit Trails & Explainability

A log that captures what was said is not an audit trail. Governance needs to know why a decision was made, who was accountable, and whether the record itself is trustworthy. These problems address the explainability and audit integrity gap.

PS-7.1

The Decision Path Auditor

Context

An AI agent declines a customer's request. The customer asks why. The audit log says: based on your application, we are unable to approve this request. That is a transcript, not an audit trail. A real audit trail for a consequential decision must capture the reasoning path: what data was considered, what tools were called, what the model's intermediate reasoning was, and what the final decision logic was.

The Challenge

Build a decision path auditor that captures the full causal chain behind an agent's consequential output — not just the final text.

What to Build

- An instrumented agent wrapper that logs at every step: input received, tool calls made with parameters and responses, any intermediate reasoning steps if available, and final output produced.
- A decision path reconstructor: given a session ID, reconstruct the full decision path as a structured timeline — input, context retrieved, tools called, reasoning steps, output.
- A decision summary generator: use an LLM to produce a plain-English explanation of the decision path suitable for a non-technical reviewer or the affected person.
- A redaction layer: auto-redact any PII from the stored decision path before it is persisted.

Success Criteria

- Decision path for a three-step agent task (retrieve, reason, decide) is correctly reconstructed with all intermediate steps.
- Decision summary is accurate and readable by a non-technical reviewer.
- PII in intermediate tool responses is correctly redacted in the stored audit record.
- Decision path is queryable by session ID, user ID, and time range.

Bonus

- Add a challenge response generator: given a decision path, generate a draft response to a regulatory challenge explaining the decision in terms of the specific data considered.

PS-7.2

The Governed Audit Log

Context

Audit logs are themselves sensitive assets. They contain user prompts which may include PII, model outputs which may contain confidential information, and decision records which may be legally sensitive. Yet audit logs are typically stored as raw ungoverned blobs. Nobody asks: who accessed the audit log? Was the PII in the log retained longer than permitted? Was the log itself tampered with?

The Challenge

Build an audit log governance layer — applying data governance policies to the audit logs themselves.

What to Build

- A log ingestion pipeline that receives raw interaction logs (prompt, response, agent ID, timestamp, user ID).
- An auto-redaction step: scan each log record for PII and replace with tokenised placeholders before storage; maintain a secure PII-to-token mapping accessible only to authorised roles.
- A retention policy engine: tag each log record with its retention category based on the agent's regulatory classification; automatically expire records past their retention window.
- A log access audit: every read of a log record is itself logged with who accessed it, when, and what they queried.
- Tamper detection: log records are hashed at write time; a verification endpoint confirms that a stored record matches its original hash.

Success Criteria

- PII in a raw log record is correctly redacted and tokenised before storage.
- A log record flagged for 90-day retention is correctly expired after 90 days in a simulation.
- Log access audit correctly records a read event.
- Tamper detection catches a manually modified log record.

Bonus

- Add a DSAR (Data Subject Access Request) handler: given a user ID, retrieve all log records referencing that user using the PII token mapping, produce a redacted summary, and mark those records for deletion on request.

Unit 8 Cost & Resource Governance

An agent that runs unchecked can exhaust a month's budget in an afternoon. Cost governance is almost entirely ignored by compliance-focused platforms. These problems address the gap between what is spent and what is accountable.

PS-8.1

Agent Budget Controller

Context

An enterprise gives its engineering team an LLM budget. The team has twelve agents running across four products. One agent enters a recursive loop and makes fifty thousand API calls overnight. The engineering team finds out when the monthly cloud bill arrives. There was no enforcement at the infrastructure layer — only post-hoc billing.

The Challenge

Build an agent-level budget controller that enforces token spend limits per agent, per team, and per time window — rejecting or rerouting requests before they exceed the budget.

What to Build

- A budget configuration system: define budgets at three levels — team (e.g., \$500 per month), agent (e.g., \$50 per month), and session (e.g., \$2 per session).
- A metering layer: intercept every LLM API call, record the token count and estimated cost, and update running totals in real time.
- Enforcement logic: when a budget is 80 percent consumed send a warning; when fully consumed reject new requests with a budget-exhausted message; when a session budget is consumed close that session.
- A budget dashboard: current spend versus limit for all active agents and teams, updated in real time.
- Model substitution on budget pressure: when an agent's preferred model budget is exhausted, automatically reroute to a cheaper model within the same provider rather than hard-blocking.

Success Criteria

- Budget correctly tracked across three agents making concurrent calls.
- Warning fires at 80 percent consumed.
- Hard block fires at 100 percent consumed.
- Session budget correctly closes a session that exceeds the per-session limit.

- Model substitution correctly reroutes to the cheaper model.

Bonus

- Add a runaway agent detector: if a single agent consumes more than 20 percent of its monthly budget within a single hour, flag it as potentially stuck in a loop and pause it for human review.

PS-8.2

Model Substitution Governance Event

Context

LLM gateways routinely substitute models for cost or availability reasons — routing to a smaller model when a larger one is over budget, or falling back to an alternative provider when one is down. This substitution is treated as a routing decision. It is not recorded as a governance event. The compliance record still shows the original model. The substituted model may have different capability limits, different guardrail behaviour, and different bias characteristics.

The Challenge

Build a model substitution governance tracker that records every model substitution as a governance event and assesses its compliance implications.

What to Build

- A substitution interceptor in an LLM gateway: detect when the model actually used differs from the model requested.
- A governance event recorder: for every substitution, create a record with requested model, actual model used, reason (cost, availability, or policy), timestamp, agent ID, and session ID.
- A substitution risk assessor: compare the capability profiles of the requested versus actual model across context window, known guardrail behaviour, and bias benchmarks if available; flag if the substitution represents a material capability downgrade.
- A compliance flag: if an agent's compliance record specifies an approved model list, a substitution to an unapproved model automatically sets a compliance flag requiring review.

Success Criteria

- Substitution is detected and recorded for all three substitution scenarios (cost, availability, policy).

-
- Risk assessment correctly flags a substitution from a large context model to a significantly smaller context model as a material downgrade.
 - Compliance flag fires when substitution lands outside the approved model list.
 - Governance event log is queryable by agent, time range, and substitution reason.

Bonus

- Add a retroactive compliance impact assessment: given a list of substitution events, compute the total number of requests served by a non-approved model and the potential compliance exposure.

Unit 9 Human-in-the-Loop & Oversight Controls

At agent scale, oversight has to be smart. Humans cannot review everything, but they must remain meaningfully in control. These problems address the gap between a binary on/off HITL switch and the graduated, accountable oversight that high-stakes AI requires.

PS-9.1

Graduated Autonomy Engine

Context

Today agents are either fully autonomous or fully gated. Neither extreme is right. A fully autonomous agent takes unnecessary risks on high-stakes actions. A fully gated agent requires human approval for trivial decisions and becomes a bottleneck that eliminates the value of automation. What is needed is a middle ground: dynamic autonomy levels that adjust based on the risk of the action being taken.

The Challenge

Build a graduated autonomy engine that dynamically scores every agent action on a risk dimension and routes it to the appropriate autonomy level: autonomous, confirm, or full review.

What to Build

- A risk scorer for agent actions covering four dimensions: reversibility (can the action be undone?), data scope (how many records or users are affected?), regulatory category (is the action in a regulated domain?), and confidence (how certain is the LLM's output?).
- A risk-to-autonomy mapping: actions below a low-risk threshold execute autonomously; actions in the medium range show a preview to the user and execute on confirmation; actions above the high threshold route to a human review queue.
- A confirmation interface (a simple CLI prompt or webhook simulation is acceptable): show the user what the agent is about to do and ask for confirm or reject.
- An audit record: every routing decision logged with the risk score breakdown.

Success Criteria

- A bulk database delete (high risk) routes to full review.
- A single record update (medium risk) routes to user confirmation.
- A read-only query (low risk) executes autonomously.
- Risk score breakdown is accurate and human-readable in the audit log.

Bonus

- Add adaptive threshold calibration: if users consistently confirm a particular action type without modification, lower its risk score automatically over time; if they consistently modify or reject it, raise it.

PS-9.2

HITL Review Queue with SLA Enforcement

Context

An agent escalates a high-risk decision to a human review queue. The reviewer is busy. The review sits pending for three days. The customer affected by the decision has been waiting. There is no SLA, no escalation, and no record that the delay happened. In a regulated context, a three-day unreviewed decision in a high-risk AI system is itself a compliance event.

The Challenge

Build a HITL review queue with governance SLA enforcement — turning the human review step into a governed, accountable process.

What to Build

- A review queue: pending agent decisions added with agent ID, action description, risk score, triggering context, submission timestamp, and assigned reviewer.
- SLA tiers based on risk level: high risk (4-hour SLA), medium risk (24-hour SLA), and low risk (72-hour SLA).
- SLA enforcement: a background monitor checks open items against their SLA; at 50 percent of SLA elapsed send a reminder; at 100 percent auto-escalate to the next reviewer tier and create a compliance event.
- Reviewer actions: approve (agent proceeds), reject (agent is blocked with reason), modify (agent proceeds with amended instruction).
- An SLA compliance report: percentage of reviews completed within SLA, by risk tier and by reviewer.

Success Criteria

- Queue correctly assigns SLA tier based on risk score.
- Reminder fires at 50 percent of SLA elapsed.
- Auto-escalation fires correctly at SLA breach and creates a compliance event record.
- All three reviewer actions work and produce correct downstream agent behaviour.
- SLA compliance report is accurate.

Bonus

- Add a review quality scorer: after a review decision, compare the reviewer's choice to the agent's original recommendation; flag systematic reviewer-agent disagreement as a signal for policy recalibration.

Unit 10 Policy Management & Governance-as-Code

Policy that lives in a separate system from the code it governs will always drift. These problems address the gap between governance intent and deployed reality — and the absence of a standard policy language that works across the entire AI governance stack.

PS-10.1

Policy-as-Code for AI Agents

Context

An agent is deployed with a policy document in a governance platform. The agent's code is versioned in Git. When the agent is updated, the policy is not. When the agent is rolled back, the policy is not rolled back either. Within weeks the running agent and its governance record are describing two different systems.

The Challenge

Build a governance-as-code framework where agent policies are defined alongside agent code, versioned together in Git, and automatically deployed and enforced as part of the CI/CD pipeline.

What to Build

- A policy definition format (YAML or HCL) that lives in the same repository as the agent code and specifies: approved model list, allowed tools and scopes, guardrail rules, HITL thresholds, data retention rules, and regulatory framework tags.
- A CI/CD integration: a GitHub Actions or equivalent pipeline step that on every agent deployment reads the policy file, validates it for completeness, and pushes it to a policy enforcement runtime.
- A policy versioning endpoint: given an agent ID and a Git commit SHA, return the exact policy that was active at that version.
- A policy drift detector: compare the currently enforced policy for an agent against the policy in the agent's latest Git commit; alert if they differ.

Success Criteria

- Policy file deploys automatically on push to the main branch.
- Policy versioning endpoint returns the correct policy for any historical commit.
- Drift detection correctly alerts when the enforced policy is manually modified out-of-band, simulating an admin change bypassing Git.
- An invalid policy file with missing required fields fails the CI step and blocks the deployment.

Bonus

- Add a policy promotion workflow: policies flow through environments (dev, staging, prod) and require a review approval step before promotion to prod, enforced as a pull request check.

PS-10.2

Dynamic Policy Rules Engine

Context

Current governance policy systems express static rules: block HAP content, deny model X to role Y. They cannot express rules such as: between 9pm and 6am require HITL for any action that modifies data; or if the current session's data context includes patient records apply HIPAA-level guardrails regardless of the agent's default policy. Governance needs to be context-aware, not just static.

The Challenge

Build a dynamic policy rules engine that evaluates governance rules against a rich runtime context — enabling time-aware, session-aware, and data-context-aware policy.

What to Build

- A context object assembled at request time containing: timestamp, user_role, session_data_classification, agent_id, action_type, data_scope_size, and previous_violations_in_session.
- A policy rule language (DSL) that can express conditions over the context object, for example: if timestamp hour is outside business hours and action type is data write then require HITL; if session data classification is PHI then apply the HIPAA strict policy; if session violations count exceeds two then suspend session.
- A rule evaluator: given the context object and the active policy, evaluate all applicable rules and return the enforcement decision.
- A policy composition system: policies can inherit from a base policy and override specific rules without replacing the whole policy.

Success Criteria

- Time-aware rule fires correctly outside business hours and not within them.
- Data classification rule correctly escalates to HIPAA-level guardrails when PHI is detected in session context.
- Violation accumulator correctly suspends a session after three violations.

-
- Policy inheritance works: a child policy inherits base rules and adds overrides; base rules that are not overridden still apply.

Bonus

- Add a policy simulation mode: given a historical session log, replay it through a proposed new policy and show which decisions would have changed — safe policy testing before deployment.